# VaR 回測與風險管理

第七週 — 財務管理：波動率、風險與人工智慧

Yi-Hao Lai

2026 Spring

# 今日學習路徑


1. 故事：監管機構來電
2. 回測的邏輯
3. Kupiec 無條件覆蓋率檢定
4. Christoffersen 條件覆蓋率檢定
5. Basel 交通號誌系統
6. 壓力測試
7. Python 實作
8. 應用：金融廳報告
9. 重點摘要

# 1

## 故事：監管機構來電

# VolTech 的挑戰

**情境**：日本金融廳要求在兩週內提交完整的回測報告。

他們要求：Kupiec 檢定、Christoffersen 檢定，以及極端情境下的壓力測試。

如果未通過，基金必須持有更多資本——可用於投資回報的資金就更少了。

**核心問題**

我們的 VaR 模型是否名副其實？如果我們宣稱 99% VaR 只有 1% 的時間被突破，我們能證明嗎？

# 今日學習目標

在本次課程結束時，你將能夠：

1. 定義 VaR 回測（Backtesting）並解釋為何監管機構要求它
2. 實作 Kupiec（POF）無條件覆蓋率檢定
3. 實作 Christoffersen 條件覆蓋率檢定
4. 解讀 Basel 交通號誌系統
5. 設計與執行壓力測試（Stress Testing）

# 2

# 回測的邏輯

# 什麼是 VaR 回測？

**回測（Backtesting）** = 將 VaR 預測值與實際損失進行比較。

定義**命中序列（Hit Sequence）**：

$$I_t = \begin{cases} 1 & \text{若} r_t < -\text{VaR}_t \quad \text{（突破）} \\ 0 & \text{其他} \end{cases}$$

一個正確模型的命中序列必須滿足：

1. **無條件覆蓋率**：突破比例 $= 1 - \alpha$
2. **獨立性**：突破事件不應群聚

# 為何兩個性質都很重要

**模型 A**：250 天中有 3 次突破，分散在一月、六月、十一月。

突破率 $= 1.2\% \approx$ 預期的 1%。

$\rightarrow$ 通過無條件覆蓋率檢定。

$\rightarrow$ 通過獨立性檢定。

**模型 B**：250 天中有 3 次突破，全部集中在崩盤期間的同一週。

相同突破率 $= 1.2\%$。

$\rightarrow$ 通過無條件覆蓋率檢定。

$\rightarrow$ 未通過獨立性檢定（群聚 !）

**關鍵洞見**

Kupiec 檢定捕捉錯誤的次數。Christoffersen 檢定捕捉群聚現象。你**兩者都需要**。

# 3

## Kupiec 無條件覆蓋率檢定

# Kupiec POF 檢定（1995）

**Kupiec 概似比**

$$LR_{uc} = -2\ln\frac{p^x(1-p)^{T-x}}{\hat{p}^x(1-\hat{p})^{T-x}} \sim \chi^2(1)$$

- $T =$ 總樣本外天數
- $x =$ 突破次數
- $p = 1 - \alpha =$ 預期突破率（例如 0.01）
- $\hat{p} = x/T =$ 觀察突破率

**決策**：若 $LR_{uc} > 3.841$（5% 顯著水準），則拒絕 $H_0$

# Kupiec 檢定：實務考量

對於 $T = 250$ 天與 99% VaR：

- 預期突破次數：$250 \times 0.01 = 2.5$
- 大約通過範圍：0 至 6 次突破
- $\geq 7$ 次突破 $\rightarrow$ 可能被拒絕

## 低檢定力警告

僅有 250 天的資料，Kupiec 檢定無法可靠地區分 1% 與 2% 的突破率。這是小樣本的根本限制，而非檢定本身的缺陷。

# 4

## Christoffersen 條件覆蓋率檢定

# 轉移矩陣

定義轉移機率：$\pi_{ij} = P(I_t = j \mid I_{t-1} = i)$

| | $I_t = 0$ | $I_t = 1$ |
|---|---|---|
| $I_{t-1} = 0$ | $n_{00}$ | $n_{01}$ |
| $I_{t-1} = 1$ | $n_{10}$ | $n_{11}$ |

**獨立性**意味著：$\hat{\pi}_{01} = \hat{\pi}_{11}$

**群聚**意味著：$\hat{\pi}_{11} > \hat{\pi}_{01}$

(一旦發生突破，更多突破可能接踵而至。)

# Christoffersen 檢定（1998）

## 條件覆蓋率檢定

$$LR_{cc} = \underbrace{LR_{uc}}_{\text{覆蓋率}} + \underbrace{LR_{ind}}_{\text{獨立性}} \sim \chi^2(2)$$

- $LR_{uc}$：Kupiec 無條件覆蓋率（同前）
- $LR_{ind}$：使用轉移次數的獨立性檢定
- 合併檢定：若 $LR_{cc} > 5.991$ 則拒絕

## 解讀

一個模型可以**通過** Kupiec（正確次數）但**未通過** Christoffersen（突破群聚）。

# 5

## Basel 交通號誌系統

# Basel 交通號誌框架

| 區域 | 突破次數（250 天） | 解讀 | 乘數 |
|---|---|---|---|
| 綠燈 | 0–4 | 可接受 | $3.0\times$ |
| 黃燈 | 5–9 | 有疑慮 | $3.4$–$3.85\times$ |
| 紅燈 | $\geq 10$ | 被否決 | $4.0\times$ |

**⚠ 財務影響**

紅燈區域 $=$ 資本乘數從 $3.0\times$ 增加到 $4.0\times$。對於風險暴露為 \$10B 的基金，這意味著 **多出 \$10B 的資本**被鎖定——無法用於投資。

# 黃燈區域細節

黃燈區域有階梯式乘數：

| 突破次數 | 乘數 |
|---|---|
| 5 | $3.40\times$ |
| 6 | $3.50\times$ |
| 7 | $3.65\times$ |
| 8 | $3.75\times$ |
| 9 | $3.85\times$ |

**關鍵洞見**：銀行有強烈動機維持在綠燈區域。VaR 模型品質具有直接的財務後果。

# 6

# 壓力測試

# 壓力測試框架

**回測** = 在正常條件下驗證模型
**壓力測試** = 在極端情境下評估表現

**歷史壓力測試**：

- 2008 全球金融危機
- 2020 COVID-19 崩盤
- 2011 歐洲債務危機
- 1997 亞洲金融危機

**假設壓力測試**：

- 市場突然下跌 5%
- VIX 從 15 飆升至 80
- 亞洲市場同步崩盤
- 利率衝擊 +300 基點

# 壓力測試結果解讀

對每個情境計算：

$$\text{VaR 超越比率} = \frac{\text{情境損失}}{\text{VaR}}$$

| 情境 | 損失 | VaR 比率 |
|---|---|---|
| 2008 GFC（最差日） | $-9.47\%$ | $3.3\times$ |
| 2020 COVID-19 | $-12.77\%$ | $4.5\times$ |
| 2010 閃崩 | $-3.87\%$ | $1.4\times$ |
| 假設 $-10\%$ | $-10.00\%$ | $3.5\times$ |

VaR 並非為危機而設計。壓力測試量化 VaR 之外的暴露。

# 7

## Python 實作

# 步驟一：VaR 回測設置

```python
import numpy as np, pandas as pd
import yfinance as yf
from arch import arch_model
from scipy import stats

sp500 = yf.download("^GSPC",
    start="2015-01-01", end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(
    prices / prices.shift(1)).dropna()

window = 1000; alpha = 0.99
n_oos = len(returns) - window
print(f"Backtest: {n_oos} days")
print(f"Expected breaches: {n_oos*0.01:.1f}")
```

# 步驟二：滾動 GJR-GARCH VaR

```python
var_bt = np.full(n_oos, np.nan)
for i in range(n_oos):
    train = returns.iloc[i:i + window]
    model = arch_model(train, vol="Garch",
        p=1, o=1, q=1, dist="t")
    res = model.fit(disp="off")
    fv = res.forecast(horizon=1)
    vol = np.sqrt(fv.variance.values[-1, 0])
    nu = res.params["nu"]
    q = stats.t.ppf(1 - alpha, df=nu)
    scale = np.sqrt((nu - 2) / nu)
    var_bt[i] = -(res.params.get("mu", 0)
                  + q * scale * vol)

breaches = returns.iloc[window:] < -var_bt
print(f"Breaches: {breaches.sum()}")
```

# 步驟三：Kupiec 檢定

```python
def kupiec_test(hits, alpha=0.99):
    T = len(hits); x = hits.sum()
    p = 1 - alpha; p_hat = x / T
    lr = -2 * (x * np.log(p / p_hat) +
        (T-x) * np.log((1-p) / (1-p_hat)))
    pval = 1 - stats.chi2.cdf(lr, df=1)
    return lr, pval

lr, pval = kupiec_test(breaches)
print(f"LR_uc = {lr:.3f}")
print(f"p-value = {pval:.4f}")
print("PASS" if pval > 0.05 else "FAIL")
```

# 步驟四：Christoffersen 檢定

```python
def christoffersen_test(hits, alpha=0.99):
    h = hits.astype(int).values
    n00=n01=n10=n11=0
    for t in range(1, len(h)):
        if h[t-1]==0 and h[t]==0: n00+=1
        elif h[t-1]==0 and h[t]==1: n01+=1
        elif h[t-1]==1 and h[t]==0: n10+=1
        else: n11+=1
    pi01 = n01/max(n00+n01,1)
    pi11 = n11/max(n10+n11,1)
    # ... compute LR_ind and LR_cc
    return lr_cc, pval

lr_cc, pval = christoffersen_test(breaches)
print(f"LR_cc = {lr_cc:.3f}, p = {pval:.4f}")
```

# 步驟五：Basel 交通號誌

```python
def basel_zone(n_breaches):
    if n_breaches <= 4:
        return "GREEN", 3.0
    elif n_breaches <= 9:
        mult = {5:3.4, 6:3.5, 7:3.65,
                8:3.75, 9:3.85}
        return "YELLOW", mult[n_breaches]
    else:
        return "RED", 4.0

last_250 = breaches.iloc[-250:]
zone, mult = basel_zone(last_250.sum())
print(f"Zone: {zone}, Mult: {mult}x")
```

# 步驟六：壓力測試

```python
scenarios = {
    "GFC 2008":      -0.0947,
    "COVID 2020":    -0.1277,
    "Flash Crash":   -0.0387,
    "Hypo -5%":      -0.05,
    "Hypo -10%":     -0.10,
}
current_var = var_bt[-1]
for name, loss in scenarios.items():
    ratio = abs(loss)*100 / current_var
    flag = "!!" if abs(loss)*100 > current_var \
        else ""
    print(f"{name:20s} {ratio:.2f}x {flag}")
```

# 8

## 應用：金融廳報告

# 金融廳回測摘要

| 指標 | 日經 225 | S&P 500 |
|---|---|---|
| 突破率 | 1.4% | 1.0% |
| Kupiec $p$ 值 | 0.265 | 1.000 |
| Christoffersen $p$ | 0.144 | 0.571 |
| Basel 區域 | 綠燈 | 綠燈 |

**結論**：GJR-GARCH 搭配 $t$ 分配在兩個指數上均通過所有監管要求。

關鍵優勢：模型每日調適——比歷史模擬法更快從危機衝擊中恢復。

# 9

# 重點摘要

# 六件必記之事

1. **回測驗證 VaR**：使用命中序列 $I_t$ 將預測與實際損失比較
2. **Kupiec 檢定**：檢驗無條件覆蓋率——總突破率是否正確？（$\chi^2(1)$）
3. **Christoffersen 檢定**：檢驗覆蓋率與獨立性——突破是否群聚？（$\chi^2(2)$）
4. **Basel 交通號誌**：綠燈（0–4）、黃燈（5–9）、紅燈（$\geq 10$），直接影響資本
5. **壓力測試補充回測**：正常條件 vs. 極端情境——兩者皆為必要
6. **GJR-GARCH + $t$**：持續達到綠燈區域，優於更簡單的模型

# 任務七：金融廳回測報告

## 交付項目

1. 使用 GJR-GARCH($t$) 對日經與 S&P 計算滾動 99% VaR
2. 對兩個指數進行 Kupiec 與 Christoffersen 檢定
3. Basel 交通號誌分類（最近 250 天）
4. 回測視覺化（報酬率 + VaR + 突破事件）
5. 壓力測試：3 個歷史情境 + 2 個假設情境
6. 致金融廳 300 字執行摘要信

**加分**：比較 3 種 VaR 方法（歷史模擬法、GARCH-常態、GJR-GARCH-$t$）並決定哪種表現最佳。

# 下週預告

## 第八週：AI 與金融

David：「QuantStar 正在推銷一套純 ML 風險系統——隨機森林、LSTM。他們宣稱準確度提高 50%。」

Dr. Lin：「以什麼衡量標準更準確？相對於什麼基準？具備什麼可解釋性？」

**主題**：機器學習於風險管理、模型可解釋性、AI + GARCH 混合方法